Purpose of this Document

This document is a **full, integrated, detailed product + systems specification** for building a **location-based realistic cat simulation app/game** for **iOS 17 and above**, intended to be handed directly to an AI coding/build system such as **OpenCode / Codex**.

Important instruction:

- **Do not simplify this concept into an MVP.**
- **Do not postpone major systems.**
- Build the architecture so **all listed systems are supported from the beginning**, even if some are shipped behind feature flags or staged rollout.
- Prioritize a **robust architecture, expandable data model, and clean simulation engine** so the full vision can exist coherently.

The product should feel like:

- a **realistic cat ecology simulation**
- a **pet ownership simulator**
- a **location-based encounter game**
- a **persistent world simulation**
- a **life/death/emotion-driven animal world**

Art direction should be **pixel-based**, but systems should be **deep and realistic**.

---

1) PRODUCT OVERVIEW

1.1 Product Concept

A **realistic cat pet simulation game** where users:

- walk around the real world
- encounter **persistent stray cats** based on real location
- feed / interact with / rescue / adopt them
- care for adopted cats at home

- influence local cat ecology through sterilization, feeding, medical care, and shelter systems
- witness births, territoriality, sickness, incidents, aging, and death

The world must feel:

- **persistent**
- **territorial**
- **probabilistic but believable**
- **emotionally consequential**

This is **not** a casual Tamagotchi clone. It should feel like a **living cat world**.

1.2 Core Experience Pillars

A. Real-world encounter realism

Users should feel like they are “running into” neighborhood cats based on actual location.

B. Emotional persistence

Cats are not disposable collectibles. Each cat should feel like an individual with:

- traits
- history
- parents
- health profile
- territory
- memory
- possible death

C. Stray ecology simulation

Stray populations should feel alive, not randomly generated every session.

D. Responsible pet care simulation

Adopting a cat creates real maintenance and emotional attachment.

E. Long-term emergent stories

Unexpected outcomes should emerge naturally:

- local cat family trees
- recurring cats
- rescue stories
- illness arcs
- death and memorials

---

2) PLATFORM / APP REQUIREMENTS

2.1 Platform

- **Primary platform**: iPhone
- **Minimum OS**: **iOS 17+**
- Build using:
  - **SwiftUI** for UI
  - native Apple frameworks where possible
  - backend should support real-time persistence and asynchronous simulation

---

2.2 Required Permissions

The app should strongly depend on:

Required / Core

- **Location permissions**
  - preferably **Always Allow**
  - needed for stray cat detection and region simulation
- **Notifications**
  - needed for encounter alerts, incidents, health warnings, etc.

Optional / Nice to have

- Motion / background refresh (if useful)
- Camera (future cat AR / memorial / sharing features)

Product rule

The app should clearly communicate that the full gameplay loop is designed around:

- **location being enabled**
- **notifications being enabled**

If these are disabled, the app should still function, but with degraded encounter behavior.

---

3) ART / PRESENTATION DIRECTION

3.1 Art Style

- **Pixel art** overall
- Cats should feel like:
  - **3D pixelated / pseudo-3D pixel sprites**
  - generated from modular appearance components

Visual goals

- Cute but not childish
- Slightly melancholic / warm / realistic
- Distinct silhouettes and coat patterns
- Should support:
  - portraits
  - idle states
  - health states
  - age changes
  - accessories / store items

3.2 Tone

Should balance:

- warmth

- realism
- responsibility
- occasional sadness
- emergent storytelling

Avoid making death / sickness feel like a joke.

---

4) GLOBAL TIME SYSTEM

4.1 Core Time Rule

In-game world progression runs at 5x real-life speed, but only for **long-term simulation systems**.

Systems that should use 5x speed

- aging
- territorial changes
- reproduction probability
- stray population changes
- probabilistic incidents
- health progression (certain types)
- disease progression
- local ecology changes
- world events

Systems that should NOT be 5x accelerated

Routine care should remain closer to real cadence:

- feeding
- litter cleaning
- play interaction
- affection / petting
- medication timing (unless medically designed otherwise)
- daily home care loop

Product intention

The game should:

- progress fast enough to feel alive
- but not punish players by requiring impossible real-life maintenance

---

4.2 Time Architecture Requirement

The system must support **multiple time domains**:

A. Real-world time

Actual timestamp

B. Simulation time

Used for world-state progression

C. Care-cycle time

Used for home cat maintenance loops
These should be stored separately to avoid design contradictions.

---

5) INITIAL USER SETUP / ONBOARDING

5.1 First-Time Setup Flow

On first launch, user should go through:

Step 1 — Welcome / concept explanation

Explain:

- cats exist in the real world around them
- home and outside are different gameplay zones
- location + notifications improve gameplay

Step 2 — Set Home Location

User must define a **home location**.

Requirements

- Search-based location selection
- Does **not** need to be current location
- User can set:
  - apartment area
  - house area
  - neighborhood center

Important note

Home location should be a **gameplay anchor**, not necessarily exact GPS.

Step 3 — Home Location Change Rules

Home location should not be infinitely changeable.

Proposed system

Allow home change by:

- special token
- cooldown
- premium item
- limited number of free changes

This prevents abuse of the home/outdoor system.

Step 4 — Set Nickname

Simple display name / player name

Step 5 — Start State

Player begins with:

- **no pet cat**
- empty home
- no stray relationships
- no cat inventory beyond basic starter options if desired

6) CORE GAME STRUCTURE

There are **two major gameplay domains**:

1. **Stray Cat World** (outside world)
2. **Pet Cat World** (inside home)

These should feel fundamentally different.

---

7) STRAY CAT SYSTEM (OUTSIDE WORLD)

7.1 Core Stray Rules

Rule A

Stray cats can only be encountered while the user is outside the home area.

Rule B

A stray cat encounter occurs only if:

- the user is in a valid outdoor region
- the region has existing or potential stray cat simulation
- encounter probability succeeds

Rule C

Stray cats should feel **persistent and local**, not globally random.

7.2 Stray Cat Encounter Loop

Encounter Flow

1. User is physically moving / located outside home
2. App determines current region / territory cell
3. Backend checks whether stray population exists there

4. If not, system may initialize that area
5. Encounter probability is rolled
6. If successful, one or more cats are selected
7. User receives a **notification**
8. Opening app presents the cat interaction screen
9. Or if the user decides to open the app to play, If encounter successful
10. User app directly presents the cat interaction

7.3 What Happens on Encounter

When a stray cat is encountered, user should be able to:

Interaction Actions

- **Observe**
- **Feed**
- **Play with**
- **Attempt trust-building**
- **Attempt rescue**
- **Adopt**
- **Take to vet** (directly or after rescue)
- **Castrate / sterilize** (through later workflow)
- **Microchip after adoption**

The encounter should not always allow all actions immediately depending on:

- trust level
- cat temperament
- cat condition
- whether the cat flees

7.4 Stray Cat Encounter Notification System

When user enters a region with encounter success:

Notification examples

- “A cat may be nearby.”
- “A stray cat has appeared in your area.”
- “You heard a faint meow nearby.”
- “A familiar cat is around.”

Notification logic should support:

- first encounter
- repeat encounter with known cat
- urgent condition encounter (injured / sick)
- weather danger encounter
- local kitten litter alert

7.5 Stray Cat Local Population Persistence

Every region should maintain a **persistent local stray population model**.

Each region should support:

- active stray cats
- dead stray cat records
- local births
- local migration
- local sterilization rate
- feeding support level
- shelter support level
- risk level
- cat density estimate

The user should feel that:

- the same cats can reappear
- some areas are cat-heavy
- some areas are cat-empty

- local ecology changes over time

7.6 Stray Cat Reproduction / Family Lineage

Stray cats should support **persistent parent-child lineage**.

Required lineage data

Each cat may store:

- mother ID
- father ID (if determinable)
- litter ID
- siblings
- offspring list
- generational ancestry (queryable)

Product intention

This allows:

- regional cat families
- recurring bloodlines
- “this cat is the daughter of a cat I used to feed” moments
- stronger emotional persistence

7.7 Stray Cat Death Risks

Stray cats may die from multiple causes.

Death categories

A. Environmental

- too cold
- too hot

- weather exposure
- insufficient food over recent period
- dehydration

B. Medical

- untreated illness
- infection
- chronic decline
- acute event

C. Incident

- fights
- accidents
- road / environment hazards
- pure bad luck events

D. Population-linked vulnerability

- kittens higher mortality
- elderly cats higher mortality
- sick / weak cats higher mortality

All death should feed into persistent history and ecology impact.

---

8) PET CAT SYSTEM (HOME WORLD)

8.1 Core Pet Rules

Rule A

Pet cats are only interactable within the user's home gameplay space.

Rule B

Other users should **not** be able to encounter or see another user's home pet cat.

Rule C

Pet ownership should feel meaningful, affectionate, and maintenance-heavy enough to feel real.

8.2 How a Cat Becomes a Pet Cat

A cat becomes a pet cat when:

- user adopts a stray cat
- future systems may include store / rescue / gift / breeding source, but adoption from stray system is core

After adoption:

- cat is removed from local stray pool
- cat becomes assigned to user account
- cat is moved into home system
- persistent ownership record is created

8.3 Pet Cat Daily Care Actions

Pet cats should require regular interaction.

Core care systems

- Feeding
- Water maintenance
- Litter / poop / pee cleanup
- Play with toys
- Petting / affection
- Sleep environment
- Grooming (optional but good)
- Medicine / supplements if needed
- Vaccination
- Vet visits

These should affect:

- health
- mood
- trust / bond
- disease risk
- accident risk
- long-term life expectancy

8.4 Leaving Home / Long Absence System

If the player will be away for a long time, they must make arrangements.

Options

A. Hire helper / sitter

Should handle:

- feeding
- water
- litter cleaning
- basic welfare

B. Prepare enough food and litter capacity

Lower cost, but carries risk

C. Use pet hotel

Higher cost, safer option

D. Potential future mixed options

- neighbor care
- smart feeder systems
- emergency contact systems

Product requirement

Neglect should have consequences if the player leaves without adequate preparation.

8.5 Pet Cat Death Risks

Pet cats may die if:

A. Neglect

- severe underfeeding
- unmanaged sickness
- poor sanitation
- repeated long-term neglect

B. Incident

- illness
- fights
- pure accidents
- home hazards

C. Age / natural decline

Natural aging and death should also exist.

Death should be emotionally weighty, not purely punitive.

---

9) CAT ENTITY MODEL (ALL CATS)

Every cat in the game must be modeled as a **persistent individual entity**.

9.1 Cat Identity Fields

Each cat should have:

- unique ID
- generated internal name / display label (if unnamed)

- user-given name (if adopted)
- creation / spawn timestamp
- origin region
- current region
- ownership state
- alive / dead status

9.2 Cat Core Characteristic Dimensions

The following characteristics should exist for all cats.

A. Breed

Can be:

- pure breed
- mixed breed
- unknown / generic domestic cat

Should affect:

- appearance
- disease tendencies
- size
- some behavioral tendencies

B. Temperament

Should affect:

- friendliness
- aggression
- skittishness
- trust gain speed
- willingness to be fed / touched / rescued

C. Age

Should affect:

- health risk
- appearance
- fertility
- energy
- mortality

D. Appearance

Should be visually generated from multiple modular traits.

E. Gender

Should affect:

- reproductive system
- some behavioral and population logic

F. Castrated / sterilized status

Should affect:

- breeding
- roaming
- fights
- population growth

G. Microchip status

Should affect:

- ownership / lost cat / rescue systems

H. Status

- stray
- pet / home cat
- possibly future statuses such as shelter / station-associated

I. Medical profile

Should include hidden and known medical state.

9.3 Cat Trait Design — Continuous vs Discrete

The system should support both.

Discrete fields (recommended)

- Breed
- Gender
- Castrated?
- Microchipped?
- Status (stray/home)
- Ear-tip marker
- Coat pattern class

Continuous / scalar fields (recommended)

- Friendliness
- Aggression
- Intelligence
- Playfulness
- Fearfulness
- Curiosity
- Stress tolerance
- Fertility tendency
- Disease susceptibility
- Survival resilience

This should allow very wide variation between cats.

9.4 Personality / Behavioral Diversity Requirements

Cats should feel highly distinct.

Examples:

- some cats are very gentle
- some are aggressive
- some are smart
- some are “dumb” / clumsy
- some are emotionally needy
- some are avoidant
- some get sick easily
- some are extremely resilient

This should influence both:

- simulation outcomes
- player emotional attachment

---

10) APPEARANCE GENERATION SYSTEM

Cats should be procedurally generated from modular appearance components.

Suggested appearance dimensions

- Fur base color
- Secondary pattern
- Tabby / tuxedo / calico / solid / etc.
- Coat length
- Eye color
- Ear shape
- Tail length / shape
- Body size
- Face structure
- Scars / injuries
- Ear-tip / TNR marker

- Age-linked visual changes
- Health-linked visual changes

Should support:

- profile portrait
- encounter portrait
- home portrait
- memorial portrait

---

11) SPAWN / ENCOUNTER SYSTEM (CRITICAL SYSTEM)

This is one of the most important systems in the entire app.

11.1 Research Requirements

The build should be structured to allow balancing based on real-world-informed parameters.

Need configurable spawn balancing for:

- probability of encountering cats in city areas
- probability in suburban areas
- probability in rural areas
- probability in sparse / “nowhere” areas
- differences by country or region if implemented

This should not be hardcoded rigidly.

11.2 Spawn Philosophy

A cat should **not** appear as a random collectible detached from place.

Instead:

- cats belong to territories

- cats are more likely to be seen in some areas than others
- cat presence depends on local ecological conditions

11.3 Region Initialization Rule

To save server cost and maintain realism:

Rule

A region should **not necessarily have precomputed cat data until a user activates it**.

Proposed behavior

- If no user has ever entered a region, that region may remain dormant
- When a user enters a region for the first time, the system may initialize:
  - local cat ecology profile
  - local cat density tendency
  - first spawnable stray cats
    This creates a persistent but scalable world.

11.4 Spawn Selection Rule

When user enters an active region:

Step 1

Determine whether the region has:

- existing active cats
- latent spawn potential
- no cat presence

Step 2

Roll encounter probability based on:

- local cat density

- time of day
- weather
- local feeding support
- cat behavior tendencies
- player movement / revisit frequency
- player-cat familiarity if applicable

Step 3

Select which cat(s) appear using weighted probability

11.5 Territorial Anchor Rule

Each stray cat should have a **territorial anchor**.

Required behavior

The cat should have the **highest probability** of being encountered in the area where it was originally established.

Example target concept

Out of all places a cat can appear:

- the **highest single chance (e.g. around 50% target weighting)** should be in its home / origin territory
- neighboring areas have reduced chance
- distant areas have low chance unless migration occurred

This creates believable cat territoriality.

11.6 Region Boundary / Territory Design

Need a region model for defining cat area boundaries.

Recommended implementation

Use a **grid / geohash / cell-based map partition system**.

Each cat can have:

- **Primary home cell**
- **Allowed roaming cells**
- **territory radius / adjacency map**

This is preferable to freeform polygon simulation for practical backend implementation.

11.7 Population Density Link to Reproduction

If an area has a **higher proportion of uncastrated cats**, then it should have:

- higher future birth probability
- higher stray population growth
- more mating behavior
- more territorial tension / fights
- potentially higher disease spread

This creates a meaningful simulation loop around sterilization.

---

12) CASTRATION / STERILIZATION SYSTEM

This system is both:

- a welfare mechanic
- a population-control mechanic
- a realism mechanic

12.1 Castration Data Fields

Each cat should support:

- castrated / sterilized status

- date of sterilization
- who performed it (system / vet / player-triggered)
- visible external marker if any

12.2 Visual Marker Research Requirement

Need support for regional / realistic sterilization markers.

Example questions to support in design

- What visual signs indicate a cat is sterilized?
- Should TNR ear-tipping be used?
- Are there different ear-cut conventions?
- Should larger / smaller ear cuts matter visually?

This should be supported as a configurable appearance marker.

12.3 Gameplay Effects of Castration

Castration / sterilization should influence:

- breeding eligibility
- roaming range
- mating behavior
- fight frequency
- territorial spread
- local population growth
- some health probabilities

12.4 Cost of Castration

The app should support a cost model for sterilization.

Possible uses:

- vet service cost
- rescue cost
- player decision-making
- stray welfare economy

Should be configurable by region / balance design.

---

13) MEDICAL / HEALTH SYSTEM

This should be one of the deepest systems in the game.

13.1 Medical Design Philosophy

The health system should feel:

- realistic
- probabilistic
- partially hidden
- discoverable through symptoms and diagnosis

It should **not** be a simplistic visible HP bar-only system.

13.2 Medical State Architecture

Each cat should have at least two layers of health data:

A. True underlying medical state (server truth)

Actual hidden conditions and body state

B. Player-known medical state

What the player currently knows / suspects / has diagnosed

This is extremely important.

13.3 Fog of War Medical Rule

Medical conditions should generally exist before they are known.

Recommended rule

A condition should be determined internally before the player sees or confirms it.

That means:

- a cat may already be ill
- symptoms may appear gradually
- player may not know immediately
- vet diagnosis can reveal the truth later

This creates realism and meaningful uncertainty.

13.4 Medical Conditions — Data Requirements

The system should support **all possible cat conditions** in a structured way.

Each condition should support:

- condition name
- category
- severity range
- acute vs chronic
- probability of occurrence
- trigger / risk factors
- incubation / onset logic
- symptom set
- symptom reveal timing
- diagnosis difficulty
- treatment options
- treatment success probability
- recurrence probability

- mortality risk
- interaction with age / breed / stray status / vaccination / sterilization / nutrition

---

13.5 Medical Probability Requirements

The design should support probability balancing for all conditions based on factors such as:

- age
- breed
- stray vs pet
- vaccination history
- sterilization status
- nutrition
- climate
- region
- stress
- injury history

This should be fully data-driven and configurable.

13.6 Treatment Difficulty / Medical Friction

Treatment should not always be straightforward.

Some cats may be difficult to treat because of:

- temperament
- trust level
- fear
- aggression
- cost
- disease complexity

This should especially matter for stray rescue workflows.

14) INCIDENT SYSTEM

Incidents should create emergent drama and realism.

14.1 Incident Categories

Possible incidents include:

- sudden illness
- injury
- fight
- escape
- weather danger
- hunger crisis
- emergency symptoms
- neighborhood cat station issue
- rescue opportunity
- kitten litter discovery

These incidents should arise from simulation state, not only scripted events.

14.2 Night Alarm / Emergency Wake System

A specific desired feature is a system where the app can wake owners up at night for urgent events.

Example use cases

- pet cat vomiting or distress
- severe health event
- escape event
- nearby stray emergency

Product note

This should be implemented carefully to avoid user annoyance.

Recommendation for implementation

Support:

- severity levels
- user preference controls
- emergency-only nighttime notifications

But the system itself should exist.

---

15) STRAY CAT HOME STATION / COMMUNITY WELFARE SYSTEM

This is a major social/ecology system and should be built into the architecture.

15.1 Concept

Users / neighbors should be able to help fund and maintain **stray cat stations**.

A station is a local support point where stray cats can:

- rest
- shelter
- feed
- survive more safely

15.2 Functional Effects of a Station

A station should affect local simulation by:

- increasing local survival rate
- reducing weather death probability
- increasing area attractiveness for local cats

- potentially increasing encounter likelihood nearby
- improving kitten survival
- reducing some hunger-linked risk

15.3 Maintenance / Cost

Stations should require upkeep.

Potential costs:

- food
- maintenance fee
- repair
- cleaning
- neighborhood funding

This can create a meaningful long-term community system.

15.4 Crowdfunding / Shared Funding System

Neighbors or local players should be able to contribute to a station.

Possible design:

- station has pooled funding
- multiple users in same region can contribute
- station status changes based on support level

Even if direct multiplayer is limited, this should be architecturally supported.

---

16) FINANCIAL / ECONOMY SYSTEM

The app should include a full economy layer.

16.1 Core Principle

The game should support both:

- **real-money purchases**
- **token / mission-based progression**

The user concept specifically includes:

- all items requiring real money
- but being much cheaper / mini-mission substitutable

This should be translated into a flexible monetization framework rather than a hardcoded one-rule economy.

16.2 Economy Design Requirements

The system should support:

- premium purchases
- in-game tokens
- mission rewards
- service costs
- welfare spending
- emergency spending
- memorial purchases
- convenience purchases

16.3 Purchasable / Payable Categories

The system should support costs for:

- pet store items
- food
- litter

- toys
- furniture
- carriers
- medicine
- vet visits
- vaccination
- sterilization
- surgery
- helper / sitter
- pet hotel
- stray station maintenance
- rescue / transport
- tomb / memorial
- cosmetic upgrades
- home location change tokens

16.4 Mission / Token System

The system should also support non-cash progression via:

- mini missions
- care streaks
- rescue milestones
- regional welfare achievements
- daily / weekly gameplay loops

This allows progression without making the game impossible for non-spenders.

---

17) DEATH / END-OF-LIFE SYSTEM

This system is emotionally central and must be handled carefully.

17.1 Death Rule

Cats can die.

This includes:

- stray cats
- pet cats
- kittens
- elderly cats
  Death must be a real, persistent part of the simulation.

17.2 Death Handling Options

The original concept raises two possible outcomes:

Option A

Player can purchase / maintain a **tomb / memorial**

Option B

All cat data is deleted from server

17.3 Recommended Implementation Requirement

The architecture should support **both**, but default to preserving meaningful history.

Strong recommendation

Do **not** hard-delete all dead cat data by default.

Instead support:

- memorialized dead cats
- archival mode
- optional cleanup if truly necessary for scale

This is essential for emotional continuity and lineage systems.

17.4 Memorial / Tomb System

Should support:

- tomb purchase
- memorial page
- portrait display
- name
- lifespan
- cause of death
- owner history
- rescue / adoption history
- parent lineage
- key life events

This can become one of the most emotionally powerful systems in the app.

---

18) STORE / SERVICE SYSTEMS

The app should support all realistic cat-related services.

18.1 Pet Store System

Should support purchase and use of:

- food
- treats
- toys
- litter
- bowls
- beds

- cat trees
- scratching items
- hygiene products
- travel products
- optional decorative items

18.2 Pet Hospital / Vet System

Should support:

- checkups
- symptom review
- diagnosis
- medication
- surgery
- sterilization
- vaccination
- emergency care
- chronic disease management

Should work for both:

- pet cats
- rescued / stray cats where applicable

---

19) LOCATION SYSTEM DESIGN

This is core to the app and must be robust.

19.1 Home vs Outside Classification

The app must reliably classify whether the player is:

- **at home**
- **outside home**

This classification drives the entire gameplay split.

19.2 Home Area Definition

Home should not be a single point only.

Need support for:

- home center point
- radius / region cell grouping
- fuzzy tolerance for GPS drift
  This avoids broken gameplay in apartment / city GPS environments.

19.3 Outdoor Encounter Eligibility

A user should only be eligible for stray encounter rolls when:

- outside home
- not in encounter cooldown if such system exists
- location confidence is acceptable

---

20) BACKEND / WORLD SIMULATION REQUIREMENTS

This app absolutely requires careful simulation design to avoid massive cost.

20.1 Do NOT Continuously Simulate Everything

The world should feel alive without requiring full-time expensive simulation.

20.2 Region Activation Optimization

The system should follow this rule:

Rule

The server records no stray cat data in an area until a user is located in that area.

Meaning

- inactive regions remain lightweight / dormant
- entering a new region may initialize its ecology
- this dramatically reduces server cost

20.3 Persistent Cat Rule

Once a stray cat has spawned:

Rule

It should continue to exist as a server-side persistent entity **until death** (or explicit archival logic).

That means cats are not just encounter events. They are actual entities.

20.4 Lazy / Event-Driven Simulation Rule

The server should **not** constantly tick every cat in real time.
Instead:

- store last known state
- store last simulation timestamp
- simulate forward when relevant
  This is one of the most important engineering rules in the entire project.

20.5 Required Lazy Simulation Logic

When a cat is next encountered / queried / affected, the system should compute outcomes based on:

- previous known state
- elapsed time
- environmental conditions
- age
- health
- region support
- random probability rolls

Examples of what may be rolled when time advances:

- survived or died
- moved territory
- became sick
- recovered
- reproduced
- got injured
- trust changed
- station dependence changed
  This allows realism without constant server burn.

---

21) DATA MODEL REQUIREMENTS

The architecture should be designed so the following entities can exist cleanly.

21.1 Core Entity Types

Minimum major entities should include:

- User
- Home
- Region / Territory Cell

- Cat
- CatLineage / parent relationships
- CatMedicalState
- CatAppearance
- CatBehaviorProfile
- EncounterEvent
- AdoptionRecord
- DeathRecord
- Memorial / Tomb
- Station / Stray Home Station
- EconomyTransaction
- InventoryItem
- ServiceRecord
- NotificationEvent
- TimeState / simulation snapshots

21.2 Important Architectural Requirement

All systems should be **data-driven**, not buried in UI logic.
That means:

- cat conditions should be configurable
- spawn weights should be configurable
- incident tables should be configurable
- region balancing should be configurable
- economy pricing should be configurable
  This is essential if OpenCode is generating scalable architecture.

---

22) USER EXPERIENCE / UX REQUIREMENTS

The app will contain a lot of system depth, so UX must remain emotionally clear.

22.1 Primary UX Goal

The app should feel:

- alive
- emotionally warm
- slightly mysterious
- systemically deep
- not cluttered or spreadsheet-like

22.2 Core User Modes

The UI should clearly separate:

A. Home Mode

For pet cat care

B. Outside / Explore Mode

For stray encounters and local ecology

C. Cat Detail Mode

For viewing a specific cat's profile, traits, history, and health

D. Service Mode

For store / vet / helper / station / memorial interactions

E. World / Region Mode

For local stray ecology and region support systems

---

23) OPEN RESEARCH / BALANCING REQUIREMENTS

These are not optional deletions; they are areas the implementation should be structured to support.

23.1 Spawn Ecology Research Areas

Need balancing inputs for:

- cat encounter probability by city/rural/empty area
- country differences if used
- regional density assumptions
- movement radius realism
- territory size realism

23.2 Castration Research Areas

Need support for:

- real-world ear-tip / ear-cut visual conventions
- sterilization sign realism
- cost realism by region

23.3 Medical Research Areas

Need support for:

- all cat medical conditions
- realistic probability weighting
- symptom progression
- diagnosis reveal logic
- treatment realism

---

24) ENGINEERING / OPENCODE IMPLEMENTATION INSTRUCTIONS

This section is specifically intended for the AI coding/build system.

24.1 Build Priorities

OpenCode should build this project with the assumption that:

- this is a **deep simulation product**, not a toy app
- architecture quality matters more than short-term simplification
- all systems described in this document should be represented in the architecture from the beginning

---

24.2 Implementation Priorities

The codebase should prioritize:

- clean modular architecture
- simulation engine separation from UI
- region-based persistence
- cat entity modeling
- strong local caching + backend sync
- robust notification handling
- expandable data tables
- realistic state transitions

24.3 Strong Build Constraints

Do NOT build this as:

- a simplistic collectible list app
- a single local-only Tamagotchi
- a stateless encounter generator
- a hardcoded one-cat-at-a-time toy prototype

It should be built as a **persistent world simulation app**.

24.4 Desired Outcome

The final product should support the feeling that:

> *"These cats existed before I saw them, and they may continue to exist after I leave."*

That emotional illusion is one of the most important success criteria in the entire app.

---

25) FINAL SUMMARY

This app must support all of the following as first-class systems:

- location-based stray cat encounters
- persistent territorial cat populations
- realistic cat individuality
- adoption and home pet simulation
- care and neglect systems
- sterilization and ecology impact
- hidden medical states and diagnosis
- incidents and emergency notifications
- community stray cat stations
- real-money + token economy
- death, memorials, and cat history persistence
- server-efficient lazy simulation

This should be treated as a **serious simulation architecture project with a warm pixel-art presentation layer**

The system should feel like a living cat world, not a shallow mobile pet app.